Complete Software Pipeline: GoPro Capture → VR Playback

From 8 synchronized GoPro cameras to immersive multi-viewpoint VR experience

This document covers every software step from pressing record to a viewer watching in their VR headset.

Pipeline Overview (High-Level)

```
┌──────────────────────────────────────────────────────────────┐
│                        PHASE 1: CAPTURE                      │
│  8 GoPros record → Sync verification → Offload to storage    │
│                        (30 min - 2 hours)                    │
└──────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────┐
│                     PHASE 2: PRE-PROCESSING                  │
│  Organize files → Color match → Lens correction → Sync align │
│                        (30 min - 1 hour)                     │
└──────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────┐
│                    PHASE 3: STITCHING (CORE)                 │
│  Load calibration → Stitch 8 cameras → Generate 360° stereo  │
│                     (2-8 hours per hour of footage)          │
└──────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────┐
│                   PHASE 4: POST-PROCESSING                   │
│  Color grade → Stabilize → Audio sync → Quality control      │
│                        (1-3 hours)                           │
└──────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────┐
│             PHASE 5: MULTI-VIEWPOINT INTEGRATION             │
│  Combine 5 rigs → Create hotspots → Build interactive player │
│                        (2-4 hours)                           │
└──────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────┐
│                        PHASE 6: ENCODING                     │
│  H.265 encoding → Spatial metadata → Audio encoding → Optimize │
│                        (1-4 hours)                           │
└──────────────────────────────────────────────────────────────┘
```

```
                                ↓
┌──────────────────────────────────────────────────────────────────┐
│                     PHASE 7: DISTRIBUTION                        │
│  Upload to platform → Generate previews → Test playback          │
│                        (30 min - 2 hours)                        │
└──────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────┐
│                       PHASE 8: PLAYBACK                          │
│  User downloads/streams → VR player → Multi-viewpoint switching  │
│                        (User experience)                         │
└──────────────────────────────────────────────────────────────────┘

TOTAL TIME: 7-20 hours for single-rig, 2-hour event (offline processing)
LIVE STREAMING: Requires real-time pipeline (different architecture)
```

PHASE 1: CAPTURE

1.1 Recording

During Event: - All 8 GoPros recording simultaneously via Camdo Blink trigger - Monitor: Check recording indicators on each camera - Monitor: Storage capacity (512GB = ~85 min per camera) - Audio: Separate ambisonic mic recording spatial audio (optional)

Files Generated (per rig, 2-hour event): - 8 × GoPro video files (.MP4 or .HEVC) - Camera 1-8: `GX010123.MP4`, `GX010124.MP4`, etc. (GoPros split files every ~10-15 min) - Total: ~360-400GB per rig - Ambisonic audio: 1 × multichannel audio file (.WAV, 4-channel or higher)

1.2 Sync Verification

Immediately after recording stops:

Manual Check: 1. Review first 10 seconds of each camera 2. Look for visual sync marker (clapper board, LED flash, hand clap) 3. Check audio waveform alignment (if using separate audio recorder)

Software Check:

```
# Example: Use ffprobe to check start timestamps
ffprobe -v quiet -show_entries format=start_time GX010123.MP4

# Compare timestamps across all 8 cameras
# Difference should be <1 second (Camdo Blink accuracy)
```

If sync is off: - Note timestamp differences - Will correct in pre-processing phase - Acceptable: ±1 second (correctable in post) - Unacceptable: >5 seconds (re-shoot or manual alignment)

1.3 Offload to Storage

Equipment: - NVMe SSD (2TB minimum) connected via Thunderbolt/USB-C - Transfer speed: ~500 MB/s - Transfer time: 400GB ÷ 0.5 GB/s = ~800 seconds = **13-15 minutes**

Workflow: 1. Remove MicroSD cards from all 8 GoPros 2. Insert into card reader (multi-slot or sequential) 3. Copy all files to organized folder structure:

```
/EventName_2025-01-14/
├── Rig1/
│   ├── Cam01/
│   │   ├── GX010123.MP4
│   │   ├── GX010124.MP4
│   │   └── ...
│   ├── Cam02/
│   ├── Cam03/
│   ├── ...
│   └── Cam08/
├── Rig2/
├── ...
├── Rig5/
└── Audio/
    └── ambisonic_recording.wav
```

4. Verify file integrity (check file sizes, play test frames)
5. Create backup copy to secondary drive (redundancy)

Total Phase 1 Time: 30 minutes - 2 hours (mostly offloading)

PHASE 2: PRE-PROCESSING

2.1 File Organization & Renaming

Problem: GoPros create sequential filenames that may overlap across cameras

Solution: Rename files to include camera ID

Script (Python example):

```python
import os
import shutil

for cam_id in range(1, 9):  # Cameras 1-8
    cam_folder = f"Rig1/Cam{cam_id:02d}/"
    for file in os.listdir(cam_folder):
        if file.endswith(".MP4"):
            # Rename GX010123.MP4 → Rig1_Cam01_GX010123.MP4
```

```
            new_name = f"Rig1_Cam{cam_id:02d}_{file}"
            os.rename(cam_folder + file, cam_folder + new_name)
```

2.2 Color Matching

Problem: Each GoPro may have slightly different white balance, exposure, color temperature

Solution: Normalize color across all cameras before stitching

Software Options:

Option A: DaVinci Resolve (Manual) - Import all 8 camera clips - Color match using ColorChecker reference (shot during calibration) - Apply LUT (Look-Up Table) to standardize - Export color-corrected clips - **Time**: 30-60 minutes manual work

Option B: Adobe Premiere Pro (Manual) - Use Lumetri Color to match cameras - Reference camera 1 as master - Match cameras 2-8 to camera 1 - **Time**: 30-60 minutes

Option C: Automated (Python + OpenCV)

```
# Pseudo-code for automated color matching
import cv2
import numpy as np

# Load first frame from each camera
frames = [cv2.imread(f"Cam{i}_frame1.jpg") for i in range(1, 9)]

# Compute average color histogram
avg_hist = compute_average_histogram(frames)

# Generate color correction LUT for each camera
for i, frame in enumerate(frames):
    lut = match_histogram_to_target(frame, avg_hist)
    apply_lut_to_all_frames(f"Cam{i}", lut)
```

Output: Color-matched video files ready for stitching

2.3 Lens Distortion Correction

Problem: GoPro fisheye lenses have radial distortion

Solution: Apply lens correction profile

Software: Built into stitching software (Mistika VR, PTGui) - GoPro Hero 12 lens profile included - Or generate custom profile during calibration phase

Note: Usually done automatically during stitching, not separate step

2.4 Sync Refinement (Frame Alignment)

Problem: Even with hardware trigger, cameras may be ±1 frame off

Solution: Align frames using audio or visual reference

Method 1: Audio Sync (if using separate recorder)

```
# Use audio peak (clap, beep) as sync point
# Align video frames to audio timestamp
ffmpeg -i Cam01.mp4 -itsoffset 0.033 -i Cam01.mp4 -map 0:v -map 1:a -c copy C
am01_synced.mp4
```

Method 2: Visual Sync (manual) - Load all 8 clips in timeline - Find visual sync marker (hand clap, LED flash) - Offset clips by frames until aligned

Method 3: Automated (SIFT feature matching)

```
# Find common features in overlapping regions
# Compute temporal offset between cameras
# Auto-align frames
```

Output: Frame-aligned video files, ±1 frame accuracy

Total Phase 2 Time: 30 min - 1 hour

PHASE 3: STITCHING (CORE PROCESS)

This is the most complex and time-consuming phase.

3.1 Camera Calibration (One-Time Setup)

Before stitching any footage, calibrate the rig once:

Intrinsic Calibration (per camera): - Determines: Focal length, lens distortion, principal point - Method: Record checkerboard pattern from various angles - Software: **OpenCV** (open-source) or **PTGui**

```
# OpenCV camera calibration
python calibrate_camera.py \
  --input checkerboard_cam01.mp4 \
  --output cam01_intrinsics.yaml
```

Output: `cam01_intrinsics.yaml` through `cam08_intrinsics.yaml`

Extrinsic Calibration (relative camera positions): - Determines: Rotation and translation between cameras - Method: Record scene with known features (calibration target) - Software: **Structure-from-Motion** (Bundler, Colmap) or manual in PTGui

Output: `rig_extrinsics.yaml` (8 camera positions/rotations relative to center)

Stereo Calibration (pairs): - Determines: Disparity-to-depth mapping for each stereo pair - Method: Compute epipolar geometry between adjacent cameras - Software: OpenCV stereo calibration

Output: `stereo_pair_1-2.yaml`, `stereo_pair_2-3.yaml`, etc.

Time: 2-4 hours (one-time, reuse for all shoots with same rig)

Importance: Good calibration = seamless stitching, bad calibration = visible seams

3.2 Stitching Software Options

Option A: Mistika VR (Professional, Recommended)

What it is: Industry-standard 360° VR stitching software - Developed by SGO (Spanish company, used in Hollywood) - Supports any camera rig (hardware agnostic) - GPU accelerated (NVIDIA CUDA) - Optical flow-based stitching (handles parallax)

Pricing: - **Mistika VR**: $990 - $1,490 perpetual license - Or subscription: $99/month

Workflow: 1. Import 8 camera clips 2. Load calibration file (or auto-detect camera positions) 3. Set stitch mode: Stereoscopic 360° 4. Define overlap zones (20-40% between adjacent cameras) 5. Enable optical flow (for moving subjects) 6. Configure output: Equirectangular, 8K stereo (7680 × 7680) 7. Render

Processing Time: - Real-time capable with high-end GPU (RTX 4090) - Typical: 2-4× real-time (1 hour footage = 2-4 hours render) - 8K output, optical flow enabled: 4-6× real-time

Pros: - ✅ Professional quality - ✅ Handles parallax (moving subjects) - ✅ GPU accelerated - ✅ Industry standard - ✅ Includes color grading, stabilization

Cons: - ❌ Expensive ($990+) - ❌ Learning curve (2-3 days to master)

Option B: PTGui Pro (Photo Stitching, Adapted for Video)

What it is: Panorama stitching software, primarily for photos - Can handle video frame sequences - Lower cost than Mistika VR - Good for static rigs

Pricing: €379 (~$400) perpetual license

Workflow: 1. Export video frames as image sequences (JPEG or TIFF) 2. Import frames into PTGui 3. Auto-detect control points (overlapping features) 4. Optimize alignment 5. Export stitched frames 6. Re-encode to video

Processing Time: - Very slow for video (frame-by-frame) - 1 hour footage @ 30fps = 108,000 frames - ~0.5-1 second per frame = 15-30 hours render time

Pros: - ✅ Lower cost ($400) - ✅ Excellent for static scenes - ✅ Precise control

Cons: - ❌ Not designed for video (slow) - ❌ No optical flow (struggles with motion) - ❌ Manual intensive

Verdict: Good for testing/prototyping, not production workflow

Option C: Open-Source (OpenCV, Hugin)

What it is: Free, open-source stitching tools

OpenCV: - Python/C++ library for computer vision - Includes stitching module - Requires programming

Hugin: - Open-source panorama stitcher (similar to PTGui) - Free, but dated interface

Workflow (OpenCV example):

```
import cv2

# Load 8 camera frames
images = [cv2.imread(f"cam{i}_frame.jpg") for i in range(1, 9)]

# Initialize stitcher
stitcher = cv2.Stitcher.create(cv2.Stitcher_PANORAMA)

# Stitch
status, pano = stitcher.stitch(images)

# Save result
cv2.imwrite("stitched_frame.jpg", pano)
```

Processing Time: - Slow (CPU-based, no GPU acceleration) - 10-20× real-time

Pros: - ✅ Free - ✅ Customizable (code-level control)

Cons: - ❌ Requires programming expertise - ❌ Slow (no GPU) - ❌ Basic quality (no optical flow) - ❌ Not production-ready

Option D: Custom Pipeline (Advanced)

Build your own using: - **OpenGL shaders** for GPU-accelerated warping - **CUDA** for parallel processing - **Optical flow** (OpenCV, custom) - **FFmpeg** for video I/O

Development Cost: $50K-100K **Only viable if**: You're processing thousands of hours, need proprietary algorithms

Verdict: Not recommended for initial product

3.3 Stitching Workflow (Detailed - Using Mistika VR)

Step 1: Project Setup

```
Mistika VR > New Project
- Name: "Broadway_Hamilton_2025-01-14"
- Camera Rig: Custom (8 cameras)
- Output Format: Stereoscopic 360°
- Resolution: 8192 × 8192 (8K)
```

Step 2: Import Media - Import all 8 camera clips (already color-matched, synced) - Mistika auto-detects timestamps, aligns clips

Step 3: Load Calibration - Import `rig_calibration.xml` (created during one-time setup) - Or use auto-calibration (AI estimates camera positions)

Step 4: Define Stitch Zones

```
Camera 1: 0° - 67.5° (center: 33.75°)
Camera 2: 22.5° - 90° (center: 56.25°)
...
Camera 8: 315° - 382.5° (wraps to 22.5°)

Overlap: 22.5° between adjacent cameras (15% of 155° FOV)
```

Step 5: Optical Flow Settings

```
Enable: Yes (for moving subjects)
Quality: High
Block Size: 16×16
Search Range: 32 pixels
```

Optical flow analyzes motion between frames, adjusts stitching to minimize parallax errors.

Step 6: Blending

```
Blend Mode: Multi-band blending (pyramid)
Feather Width: 10% of overlap zone
Color Match: Auto (between adjacent cameras)
```

Step 7: Stereoscopic Settings

```
Left Eye: Cameras 1, 2, 3, 4, 5, 6, 7, 8 (stitched as left view)
Right Eye: Offset by stereo baseline (78.5mm)

Stereo Pair Formation:
- Pair 1-2: 0° - 45° viewing angle
- Pair 2-3: 45° - 90°
- ...
- Pair 8-1: 315° - 360°

Disparity Mapping: Auto-calculated from calibration
```

Step 8: Output Settings

```
Format: Equirectangular
Projection: Stereoscopic (side-by-side or top-bottom)
Resolution: 8192 × 4096 (left) + 8192 × 4096 (right) = 8192 × 8192 total
Codec: ProRes 422 HQ (for intermediate, high quality)
Frame Rate: 30fps (match source)
Color Space: Rec.709 or Rec.2020 (HDR)
```

Step 9: Render

```
Destination: /Renders/Broadway_Hamilton_Stitched.mov
Hardware: GPU (NVIDIA RTX 4090 recommended)
Render Time: 2-4 hours for 2-hour source footage
```

Output Files: - `Broadway_Hamilton_Stitched.mov` (stereo equirectangular, 8K, ~200GB ProRes)

3.4 Quality Control (QC)

After stitching, review in VR headset:

1. Load stitched video in VR player (Quest 3, Vision Pro, etc.)
2. Check for:
   - ✅ Seams (should be invisible)
   - ✅ Parallax errors (objects "breaking" at seam)
   - ✅ Color matching (consistent across 360°)
   - ✅ Stereo depth (comfortable, not eye-strain)
   - ✅ Temporal consistency (no flickering)
3. If issues found:
   - Adjust stitching parameters
   - Re-render affected sections

Critical: QC in VR headset, not on flat monitor (can't see issues properly)

Total Phase 3 Time: 2-8 hours (mostly automated rendering)

PHASE 4: POST-PROCESSING

4.1 Color Grading

Software: DaVinci Resolve or Adobe Premiere Pro

Workflow: 1. Import stitched 360° video 2. Apply primary color correction: - Exposure adjustment - Contrast enhancement - Saturation (subtle, avoid oversaturation in VR) 3. Apply secondary corrections: - Sky color boost - Skin tone correction - Stage lighting balance 4. Apply creative LUT (optional, for artistic look)

VR-Specific Considerations: - Don't over-saturate (causes nausea in VR) - Boost shadows (headsets have limited dynamic range) - Test in headset during grading

Time: 30-60 minutes

4.2 Stabilization

Problem: Even tripod-mounted rigs have micro-vibrations (floor movement, wind)

Solution: 360° stabilization

Software: Mistika VR (built-in) or GoPro Fusion Studio

Workflow: 1. Analyze gyro metadata (if embedded by GoPros) 2. Compute camera motion 3. Apply inverse motion (warp video to compensate) 4. Crop edges (stabilization reduces usable frame)

Settings:

```
Stabilization Strength: Medium (avoid over-smoothing)
Crop Factor: 5-10% (lose minimal resolution)
Horizon Lock: Yes (keep horizon level)
```

Time: Auto-processed during stitching, or 15-30 minutes as separate step

4.3 Audio Sync & Spatial Audio

If using separate ambisonic microphone:

Workflow: 1. Import ambisonic audio (4-channel .WAV, AmbiX format) 2. Sync to video (use clapper board or waveform matching) 3. Verify audio aligns with visuals (check lip sync)

If using GoPro audio: - Extract stereo audio from one GoPro (usually camera 1) - Or mix audio from multiple cameras (for better coverage) - Convert to spatial audio (optional, using plugin)

Spatial Audio Encoding (Ambisonic to VR format):

```
# Convert AmbiX to VR spatial audio (Facebook 360 format)
fb360-encoder \
  --input ambisonic_4ch.wav \
  --output spatial_audio.wav \
  --format TBE  # Two Big Ears format for VR
```

Software: - **Facebook 360 Spatial Workstation** (free plugin for DAWs) - **Google Resonance Audio** (open-source) - **DearVR** (paid, high-quality)

Time: 30-60 minutes

4.4 Quality Control Review

Full playback test in VR headset: - Watch entire video (or representative samples) - Check: Image quality, stitching seams, color, audio sync, stereo comfort - Take notes of any issues - Fix and re-export if needed

Time: 30 minutes - 1 hour (depending on content length)

Total Phase 4 Time: 1-3 hours

PHASE 5: MULTI-VIEWPOINT INTEGRATION

For multi-rig captures (5 rigs = 5 viewpoints), create interactive experience with hotspot switching.

5.1 Viewpoint Organization

You now have 5 stitched videos (one per rig):

```
/FinalVideos/
├── Viewpoint_1_FrontRow.mov (8K stereo, ProRes)
├── Viewpoint_2_Balcony.mov
├── Viewpoint_3_StageLeft.mov
├── Viewpoint_4_StageRight.mov
└── Viewpoint_5_Backstage.mov
```

Each video: ~200GB (ProRes 422 HQ, 2 hours, 8K)

5.2 Interactive Player Development

Need custom VR player that allows hotspot-based viewpoint switching.

Option A: Unity-Based Player (Recommended)

Development Steps:

1. **Unity Project Setup**
   - Create new Unity project (3D, VR template)
   - Install XR Plugin (for Quest, Vision Pro, SteamVR support)
   - Install Video Player API
2. **Load 360° Videos**

```
// Unity C# script
VideoPlayer videoPlayer1 = gameObject.AddComponent<VideoPlayer>();
videoPlayer1.url = "/Videos/Viewpoint_1_FrontRow.mp4";

// Render to skybox (sphere around user)
```

```
videoPlayer1.renderMode = VideoRenderMode.MaterialOverride;
videoPlayer1.targetMaterialProperty = "_MainTex";
```

3. **Create Hotspots**

```
// Place invisible 3D sphere at viewpoint location
GameObject hotspot = GameObject.CreatePrimitive(PrimitiveType.Sphere);
hotspot.transform.position = new Vector3(5, 0, 0); // Position in 3D space

// Detect gaze (user looks at hotspot for 2 seconds)
if (GazeDetected(hotspot, 2.0f)) {
    SwitchViewpoint(2);  // Switch to Viewpoint 2
}
```

4. **Viewpoint Switching**

```
void SwitchViewpoint(int viewpointID) {
    // Fade out current video
    FadeOut(0.5f);

    // Load new video
    videoPlayer.url = GetViewpointURL(viewpointID);
    videoPlayer.Play();

    // Fade in
    FadeIn(0.5f);

    // Maintain user's head orientation
    // (don't reset where they're looking)
}
```

5. **UI Overlays**
   - Minimap showing other viewpoints
   - Labels: “Front Row”, “Balcony”, etc.
   - Visual indicators (pulsing icons) for available hotspots
6. **Build & Deploy**
   - Build for Meta Quest (APK)
   - Build for Apple Vision Pro (visionOS app)
   - Build for SteamVR (PC VR)

Development Time: 100-200 hours (~$15K-30K) **Or use existing SDK**: Headjack Unity SDK ($500-1000/year license)

Option B: Unreal Engine - Similar workflow to Unity - Better graphics, more complex

Option C: Web-Based (Three.js) - VR experience in web browser - Uses WebXR API - Lower quality, easier distribution (no app install)

5.3 Metadata & Navigation Map

Embed metadata in player:

```
{
  "title": "Hamilton - Broadway Musical VR",
  "duration": "7200",
  "viewpoints": [
    {
      "id": 1,
      "name": "Front Row Center",
      "position": {"lat": 0, "lon": 0, "elevation": 1.6},
      "videoFile": "Viewpoint_1_FrontRow.mp4",
      "thumbnailFile": "thumb_1.jpg"
    },
    {
      "id": 2,
      "name": "Balcony Overview",
      "position": {"lat": 15, "lon": 0, "elevation": 8},
      "videoFile": "Viewpoint_2_Balcony.mp4",
      "thumbnailFile": "thumb_2.jpg"
    },
    ...
  ]
}
```

Overhead map (optional): - 2D floor plan of venue - Icons showing rig positions - User can select from map instead of hotspots

5.4 Testing & QC

Test on all target platforms: - Meta Quest 3 - Apple Vision Pro - SteamVR (Valve Index, HTC Vive) - PlayStation VR2 (if targeting)

Check: - ✅ Video playback smooth (no stuttering) - ✅ Hotspots visible and responsive - ✅ Switching smooth (1-2 second transition) - ✅ Audio continuous across switches - ✅ Head orientation preserved during switch - ✅ No crashes or bugs

Total Phase 5 Time: 2-4 hours (if using pre-built SDK) or 2-3 weeks (custom development)

PHASE 6: ENCODING & OPTIMIZATION

Current state: 5 × 200GB ProRes files (1TB total) - too large for distribution

Goal: Compress to deliverable format (streaming or download)

6.1 Video Encoding

Software: FFmpeg (command-line) or Adobe Media Encoder

Target Specs (Meta Quest 3 recommended): - Resolution: 8192 × 4096 (stereo side-by-side) or 7680 × 7680 (equirect stereo) - Codec: H.265 (HEVC) - Bitrate: 100 Mbps (high quality) or 50 Mbps (standard) - Frame Rate: 30fps or 60fps (match source)

FFmpeg Command:

```
ffmpeg -i Viewpoint_1_FrontRow.mov \
  -c:v libx265 \
  -preset slow \
  -crf 18 \
  -pix_fmt yuv420p10le \
  -tag:v hvc1 \
  -movflags +faststart \
  -b:v 100M \
  -maxrate 100M \
  -bufsize 200M \
  -s 8192x4096 \
  -r 30 \
  -c:a aac \
  -b:a 320k \
  Viewpoint_1_FrontRow_VR.mp4
```

Settings Explained: - `-c:v libx265`: H.265 codec (best compression for VR) - `-crf 18`: Quality (18 = visually lossless, 23 = high quality) - `-preset slow`: Better compression (takes longer) - `-b:v 100M`: Bitrate 100 Mbps (high quality for 8K) - `-movflags +faststart`: Enable streaming (video starts before full download) - `-c:a aac -b:a 320k`: Audio codec & bitrate

Encoding Time: - 2-hour video @ 100 Mbps: ~2-4 hours encode time (CPU: Ryzen 9 or Intel i9) - With GPU encoding (NVENC): ~1-2 hours

Output File Size: - 2 hours @ 100 Mbps = ~90GB per viewpoint - 5 viewpoints = 450GB total - Still large, but 50% reduction from ProRes

6.2 Adaptive Bitrate Encoding (For Streaming)

For streaming, create multiple quality levels:

```
# High quality (100 Mbps) - for fast connections
ffmpeg -i input.mov -b:v 100M output_high.mp4

# Medium quality (50 Mbps) - for average connections
ffmpeg -i input.mov -b:v 50M output_medium.mp4

# Low quality (25 Mbps) - for slow connections
ffmpeg -i input.mov -b:v 25M output_low.mp4
```

Player automatically selects based on user's bandwidth (HLS or DASH protocol)

6.3 Spatial Audio Encoding

If using spatial audio:

```
# Encode spatial audio with video
ffmpeg -i video.mp4 -i spatial_audio.wav \
  -c:v copy \
  -c:a aac \
  -b:a 320k \
  -metadata:s:a:0 title="Spatial Audio" \
  -metadata:s:a:0 ambisonic_order=1 \
  output_with_spatial_audio.mp4
```

Ambisonic order: - 1st order: 4 channels (AmbiX) - 2nd order: 9 channels (higher precision) - 3rd order: 16 channels (professional)

6.4 Spatial Media Metadata Injection

For YouTube, Facebook, Quest Store, inject spatial metadata:

Google Spatial Media Metadata Injector:

```
# Download: https://github.com/google/spatial-media
python spatial-media-metadata-injector.py \
  --inject \
  --stereo=top-bottom \
  input.mp4 \
  output_spatial.mp4
```

Metadata Tags: - `Spherical`: true - `Stitched`: true - `StitchingSoftware`: Mistika VR - `ProjectionType`: equirectangular - `StereoMode`: top-bottom (or left-right)

Why needed: Players (YouTube, Facebook, VR apps) use metadata to properly display 360° video

6.5 Thumbnail Generation

Create preview thumbnails (for video player UI):

```
# Extract frame at 30 seconds
ffmpeg -i Viewpoint_1.mp4 -ss 00:00:30 -vframes 1 thumbnail_1.jpg
```

Generate for each viewpoint (5 thumbnails total)

Total Phase 6 Time: 1-4 hours (mostly automated encoding)

PHASE 7: DISTRIBUTION

7.1 Hosting Options

Option A: Self-Hosted (AWS S3 + CloudFront CDN)

Setup: 1. Upload videos to AWS S3 bucket 2. Enable CloudFront CDN (fast global delivery) 3. Set permissions (public or authenticated)

Cost: - Storage: $0.023/GB/month → 450GB = ~$10/month - Bandwidth: $0.085/GB transferred → 100 downloads of 90GB = ~$765 - Total: ~$775/month for 100 downloads

Pros: - ✅ Full control - ✅ No platform restrictions - ✅ Can sell directly

Cons: - ❌ You handle infrastructure - ❌ No built-in player - ❌ Higher cost at scale

Option B: VR Distribution Platforms

Meta Quest Store: - Submit app to Quest Store - Users download via store - Meta takes 30% revenue cut - Pros: Large user base, trusted platform - Cons: Approval process, revenue share

SideQuest / App Lab: - Alternative distribution for Quest - Easier approval - Pros: Simpler, lower barrier - Cons: Smaller audience

SteamVR: - Submit to Steam - Users download via Steam - Steam takes 30% cut - Pros: PC VR audience, established ecosystem

Apple Vision Pro App Store: - Submit visionOS app - Apple takes 30% cut - Pros: Premium audience, high-quality platform

YouTube VR / Facebook 360: - Upload to platforms (free) - Pros: Free hosting, large audience - Cons: Lower quality (compression), ad-supported, no multi-viewpoint

Option C: Direct Download (Vimeo, Dropbox)

Vimeo Pro: - Upload high-quality videos - Users stream or download - Cost: $20-$75/month - Pros: Good quality, simple - Cons: File size limits, not VR-optimized

7.2 Upload Process

Example: AWS S3 Upload

```
# Install AWS CLI
aws configure

# Upload all viewpoint videos
aws s3 cp Viewpoint_1_FrontRow_VR.mp4 \
  s3://my-vr-bucket/events/hamilton/ \
  --storage-class STANDARD \
```

```
  --metadata "event=Hamilton,date=2025-01-14,venue=Broadway"

# Repeat for all 5 viewpoints
```

Upload Time: - 450GB @ 100 Mbps upload = 10-12 hours - Can upload in parallel (reduce to 3-4 hours)

7.3 Player Distribution

Unity VR App: - Build APK (Quest), IPA (Vision Pro), EXE (SteamVR) - Upload to respective stores - Users download app, then download videos within app

Or Web-Based: - Host WebXR player on website - Users visit URL, videos stream directly - No app install needed (lower barrier)

7.4 Testing & QC (Final)

Test end-to-end user experience: 1. User finds content (store, website, link) 2. User downloads/streams 3. User launches VR player 4. User watches, switches viewpoints 5. User completes experience

Check: - ✅ Download/stream speed acceptable - ✅ Video quality maintained - ✅ Audio sync perfect - ✅ Viewpoint switching smooth - ✅ No crashes, buffering issues

Total Phase 7 Time: 30 minutes - 2 hours (plus upload time)

PHASE 8: PLAYBACK (USER EXPERIENCE)

8.1 User Workflow

Scenario 1: App-Based (Quest Store) 1. User downloads "Broadway VR" app from Quest Store 2. Launches app in VR headset 3. Browses available shows (Hamilton, Wicked, etc.) 4. Selects "Hamilton - January 14, 2025" 5. Downloads video (90GB, ~20 min on fast WiFi) 6. Starts playback in Front Row viewpoint 7. Looks around, sees "Switch to Balcony" hotspot 8. Gazes at hotspot for 2 seconds → smooth fade → now in Balcony view 9. Continues watching, switching viewpoints at will 10. Pauses, resumes, rewinds as desired

Scenario 2: Web-Based (WebXR) 1. User visits yoursite.com/vr/hamilton 2. Clicks "Watch in VR" button 3. Browser prompts: "Enter VR?" → Accept 4. Video streams directly (no download) 5. Same interactive experience as app 6. Can watch on Quest, Vision Pro, desktop (non-VR mode)

8.2 Player Features

Must-Have: - ✅ Play/pause - ✅ Seek (scrub timeline) - ✅ Volume control - ✅ Viewpoint switching (hotspots or menu) - ✅ Head orientation (rotate view with head movement)

Nice-to-Have: - ✅ Rewind 10s / Forward 10s - ✅ Playback speed (0.5×, 1×, 1.5×) - ✅ Subtitles/closed captions - ✅ Multi-language audio tracks - ✅ Social viewing (watch with friends, avatars) - ✅ Screen recording (capture clips)

8.3 Technical Requirements (User's Headset)

Meta Quest 3: - Resolution: 2064 × 2208 per eye (4.5MP per eye) - Can display 8K 360° video (2-3 pixels per degree) - Acceptable quality ✅

Apple Vision Pro: - Resolution: 3660 × 3200 per eye (11.7MP per eye) - Higher pixel density, 8K looks great ✅

SteamVR (Valve Index, HTC Vive Pro 2): - Varies, 1440 × 1600 to 2448 × 2448 per eye - 8K sufficient ✅

Bandwidth Requirements: - 100 Mbps stream: Requires 100+ Mbps internet - 50 Mbps stream: Requires 50+ Mbps internet - Most users: 50 Mbps sufficient for high quality

8.4 User Experience Quality Factors

What makes or breaks the experience:

✅ **Image Quality**: 8K crisp, no compression artifacts ✅ **Stereo Depth**: Comfortable, natural (proper IPD, 65mm baseline) ✅ **Stitching Seams**: Invisible (good calibration, optical flow) ✅ **Audio Sync**: Perfect lip-sync, spatial audio enhances immersion ✅ **Frame Rate**: 30fps minimum, 60fps smooth ✅ **Viewpoint Switching**: Fast (<1 second), smooth fade ✅ **Latency**: Low (for interactive hotspots) ✅ **Comfort**: No motion sickness (stable rig, good stabilization)

LIVE STREAMING PIPELINE (ALTERNATIVE)

For live streaming (vs VOD), pipeline is different:

Real-Time Stitching

Requirements: - Stitching must happen in real-time (≤33ms latency @ 30fps) - GPU-accelerated mandatory (NVIDIA RTX 4090 or better) - Multi-GPU rig for 5 simultaneous viewpoints

Workflow:

```
8 GoPros → Camdo Blink sync → Live capture cards (HDMI) →
PC with Mistika VR (real-time mode) → Stitch @ 30fps →
H.265 encoder (NVENC) → RTMP stream → CDN → Users
```

Hardware Requirements: - PC: Dual RTX 4090 GPUs, Ryzen 9 7950X, 128GB RAM - Capture: 8 × HDMI capture cards (Blackmagic DeckLink) - Cost: ~$15,000 per rig

Software: - Mistika VR (real-time mode) - OBS Studio (streaming) - Or custom pipeline (CUDA, OpenGL)

Challenges: - Very GPU-intensive (may need multi-GPU) - Limited to 30fps (60fps requires 2× power) - Network bandwidth: 100 Mbps × 5 viewpoints = 500 Mbps upload

Processing Latency: - Capture: 33ms (1 frame) - Stitching: 33ms (1 frame, if GPU fast enough) - Encoding: 33ms (1 frame) - Network: 100-500ms (CDN to user) - **Total: 200-600ms** (acceptable for live events)

Not Yet Implemented: Requires R&D, but technically feasible

SOFTWARE COST SUMMARY

Software	Type	Cost	Purpose
Mistika VR	Stitching	$990-1,490 or $99/month	Core stitching (recommended)
DaVinci Resolve Studio	Color Grading	$295 one-time	Color correction, grading
Adobe Creative Cloud	Post-Production	$60/month	Premiere, After Effects (optional)
Unity Pro	Player Development	$185/month (per seat)	VR player app
FFmpeg	Encoding	Free	Video encoding, transcoding
Google Spatial Media	Metadata	Free	VR metadata injection
AWS S3 + CloudFront	Hosting	~$10-1000/month	Video hosting, CDN
OpenCV	Calibration	Free	Camera calibration (one-time)
Headjack Unity SDK	Player SDK (optional)	$500-1000/year	Pre-built multi-viewpoint player
Total (First Year)		~$4,500-7,000	Software licenses + hosting

Annual Recurring: ~$2,500-4,000 (subscriptions + hosting)

PROCESSING TIME SUMMARY

For 2-hour event, single rig (1 viewpoint):

Phase	Time	Can Parallelize?
Capture	2 hours	N/A (real-time)
Offload	15 minutes	No
Pre-processing	30-60 min	Yes (automated scripts)
Stitching	2-8 hours	No (GPU bottleneck)
Post-processing	1-3 hours	Partially
Encoding	1-4 hours	No (CPU/GPU bottleneck)
Upload	2-4 hours	Yes (parallel uploads)
Total	**7-20 hours**	

For 5-rig capture (5 viewpoints):

- Stitching: 5 rigs × 4 hours = 20 hours (or 4 hours if 5 GPUs in parallel)
- Encoding: 5 rigs × 2 hours = 10 hours (or 2 hours if parallel)
- **Total: 15-30 hours for multi-viewpoint experience**

Turnaround Time: - **Standard**: 24-48 hours (overnight processing) - **Rush**: 12-18 hours (with parallel GPUs, premium pricing) - **Live**: Real-time (requires different architecture)

HARDWARE REQUIREMENTS (Processing Workstation)

Minimum (for prototyping): - CPU: Intel i7-12700K or AMD Ryzen 7 5800X - GPU: NVIDIA RTX 3080 (10GB VRAM) - RAM: 64GB DDR4 - Storage: 2TB NVMe SSD + 10TB HDD backup - Cost: ~$2,500

Recommended (for production): - CPU: Intel i9-13900K or AMD Ryzen 9 7950X - GPU: NVIDIA RTX 4090 (24GB VRAM) - RAM: 128GB DDR5 - Storage: 4TB NVMe SSD (RAID 0) + 20TB HDD backup - Network: 10 Gbps Ethernet (for fast file transfers) - Cost: ~$5,500

Professional (for live streaming, multi-rig): - CPU: AMD Threadripper Pro 5995WX (64 cores) - GPU: 2× NVIDIA RTX 4090 (SLI/NVLink) - RAM: 256GB DDR5 ECC - Storage: 8TB NVMe SSD (RAID 0) + 40TB NAS - Capture Cards: 8× Blackmagic DeckLink 4K (HDMI input) - Cost: ~$25,000

QUALITY CONTROL CHECKPOINTS

Throughout pipeline, check quality at these points:

1. **Post-Capture**: Verify all 8 cameras recorded, files intact
2. **Post-Sync**: Verify frames aligned (±1 frame)
3. **Post-Stitching**: Check seams, parallax, color in VR headset
4. **Post-Grading**: Verify colors natural, not oversaturated
5. **Post-Encoding**: Check compression artifacts, bitrate
6. **Pre-Upload**: Test playback locally (all viewpoints)
7. **Post-Upload**: Test streaming speed, buffering
8. **User Acceptance**: Have client review in VR before final delivery

Each checkpoint: 5-15 minutes **Critical**: Catch errors early (cheaper to fix before encoding)

BOTTLENECKS & OPTIMIZATION

Slowest steps: 1. **Stitching** (2-8 hours) → Solution: Better GPU, parallel rigs 2. **Encoding** (1-4 hours) → Solution: GPU encoding (NVENC), parallel 3. **Uploading** (2-4 hours) → Solution: Faster internet, parallel uploads

Optimizations: - **Cloud Rendering**: Upload raw footage to AWS, render on cloud GPUs (expensive but fast) - **Multi-GPU**: Stitch 5 rigs simultaneously (5× GPUs) → 4 hours instead of 20 hours - **Proxy Workflow**: Stitch at 4K first for quick QC, then full 8K render - **Automated Scripts**: Batch processing (set and forget overnight)

Goal: 24-hour turnaround (capture evening event, deliver next day)

CUSTOM DEVELOPMENT NEEDS

What requires custom code (vs off-the-shelf):

1. **Multi-Viewpoint Player** (Unity or Web-based): $15K-30K development
2. **Automated Pre-Processing Scripts**: $5K-10K (Python, file management)
3. **Cloud Rendering Pipeline**: $20K-40K (if scaling to cloud)
4. **Live Streaming Pipeline**: $50K-100K (real-time stitching, significant R&D)

What's off-the-shelf: - ✅ Stitching (Mistika VR) - ✅ Color grading (DaVinci Resolve) - ✅ Encoding (FFmpeg) - ✅ Hosting (AWS, Vimeo) - ✅ Calibration (OpenCV, PTGui)

Recommendation: Start with off-the-shelf, custom-develop player ($15K-30K), defer live streaming to Phase 2

FINAL CHECKLIST: SOFTWARE PIPELINE READINESS

Before you build hardware, ensure you have software covered:

Phase 1 (Prototype): - ☐ Mistika VR license ($990) or trial - ☐ DaVinci Resolve (free version OK for testing) - ☐ FFmpeg installed - ☐ Unity + VR SDK (free for testing) - ☐ Processing workstation (≥RTX 3080) - ☐ Test workflow on sample GoPro footage (before building rig)

Phase 2 (Production): - ☐ Full Mistika VR license - ☐ DaVinci Resolve Studio - ☐ Unity Pro license - ☐ AWS account + S3 setup - ☐ Custom player developed - ☐ Production workstation (RTX 4090) - ☐ Automated scripts for pre-processing - ☐ QC procedures documented

Total Software Investment (Phase 1): ~$2,000 (Mistika VR + workstation upgrades) **Total Software Investment (Phase 2)**: ~$20,000-40,000 (licenses + custom development + hosting)

CONCLUSION

The software pipeline is complex but achievable using mostly off-the-shelf tools.

Key Takeaways: 1. **Stitching is the core challenge** (2-8 hours per rig) → Use Mistika VR 2. **Multi-viewpoint player requires custom dev** ($15K-30K) → Use Unity 3. **Processing time: 24-48 hours** per event (acceptable for VOD) 4. **Live streaming requires major R&D** ($50K-100K) → Phase 2 feature 5. **Software costs: ~$5K-10K first year** (licenses + hosting)

You have a clear path from GoPro footage to deliverable VR experience.

Next step: Test the stitching workflow with sample footage before building hardware rig.